Software Requirements Specification

POS Android Application

Point-of-Sale System for Handheld Terminal (POS-T8-3)

Version	1.0
Status	Draft
Date	June 07, 2026
Target Device	Handheld POS Terminal – Model POS-T8-3
Platform	Android (Native App)
Database	SQLite (on-device)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for a Point-of-Sale (POS) Android application designed to run on a handheld POS terminal (Model: POS-T8-3). The document is intended to serve as a reference for developers, testers, and stakeholders throughout the software development lifecycle.

1.2 Scope

The application — hereinafter referred to as **SmartPOS** — is a self-contained retail management and point-of-sale system. It enables store employees to:

- Manage a product catalog (items, categories, prices, discounts, barcodes).
- Conduct sales transactions by scanning barcodes or manually selecting items.
- Print receipts and invoices using the device's built-in 80mm thermal printer.
- View real-time sales analytics and generate date-range reports.

All data is stored locally on the device using SQLite. No internet connection is required for core operations.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
POS	Point of Sale – the system where a retail transaction is completed.
SRS	Software Requirements Specification.
SQLite	A lightweight, file-based relational database embedded in the application.
SKU	Stock Keeping Unit – a unique identifier for each product variant.
ESC/POS	A thermal printer command language used to format receipts.
CRUD	Create, Read, Update, Delete – basic database operations.
UI	User Interface.
FR	Functional Requirement.
NFR	Non-Functional Requirement.

1.4 References

- ESC/POS Command Reference – Epson Corporation.
- Android Developer Documentation – developer.android.com.
- SQLite Documentation – sqlite.org.

- POS-T8-3 Hardware Specification Sheet (physical box label, reviewed June 2026).

1.5 Overview

Section 2 describes the overall system context and constraints. Section 3 covers functional requirements organized by feature module. Section 4 covers non-functional requirements. Section 5 presents use cases. Section 6 defines data models. Section 7 addresses hardware interfaces.

2. Overall Description

2.1 Product Perspective

SmartPOS is a standalone Android application that operates entirely on a handheld POS terminal. It does not depend on a remote server or cloud connectivity for its core functionality. The application interfaces directly with:

- The device's **built-in barcode scanner** via Android Intent or direct serial/HID input.
- The device's **built-in 80mm thermal printer** via ESC/POS commands over USB or internal SDK.
- An on-device **SQLite database** for persistent data storage.

2.2 Target Hardware

Model	POS-T8-3 Handheld POS Terminal
OS	Android (version to be confirmed with device vendor)
RAM / ROM	3 GB RAM + 16 GB Internal Storage
Display	8-inch IPS, 800×1280 resolution
Printer	Built-in 80mm Thermal Printer, 90mm/s print speed, ESC/POS compatible
Connectivity	4G LTE + Wi-Fi (All Netcom)
Battery	6400 mAh Li-Ion
Extras	Optional fingerprint reader, optional 58mm paper roll support

2.3 Product Functions (Summary)

The application is divided into two primary operational modes:

- **Back-Office / Dashboard Mode** – Product and category management, analytics, and report generation.
- **Cashier / Sales Mode** – Transaction processing, barcode scanning, and receipt printing.

2.4 User Classes and Characteristics

User Class	Description	Access Level
Employee / Cashier	Operates the sales terminal, scans items, processes payments, prints receipts.	Sales mode; view-only analytics
Store Manager / Admin	Manages products, categories, purchase orders, discounts; views all analytics; generates reports.	Full access to all modules

2.5 Operating Environment

- The application runs natively on Android on the POS-T8-3 device.
- SQLite database is stored in the application's private internal storage directory.
- Internet connectivity is optional; all core features function offline.
- The thermal printer is accessed via the device's internal SDK or a USB serial port.
- The barcode scanner input is handled as keyboard/HID input or a dedicated scanner SDK.

2.6 Design and Implementation Constraints

- The application must be a single APK installable on the POS-T8-3 device.
- Storage is limited to 16 GB; the database must be managed efficiently.
- No cloud sync is required; all data persists locally in SQLite.
- Thermal printing must follow ESC/POS standard commands for 80mm paper width.
- Barcode generation must support at least Code128 and QR Code formats.
- The UI must be optimized for touch on an 8-inch screen.

2.7 Assumptions and Dependencies

- The device vendor provides an Android SDK or documented serial API for printer and scanner access.
- Paper roll used is 80mm width thermal paper.
- The system uses one designated primary currency. One additional secondary currency is supported via a manually configured exchange rate; see Section 3 (FR-CUR) for full requirements.
- Payment processing (card readers, payment gateways) is out of scope; the app records cash sales only.

3. Functional Requirements

3.1 Category Management

The system shall allow administrators to manage product categories to organize the catalog.

ID	Description	Priority
FR-CAT-01	The system shall allow creating a new category with a name, optional description, and an optional parent category selected from existing root-level categories.	High
FR-CAT-02	The system shall allow editing the name, description, color, and parent of an existing category.	High
FR-CAT-03	The system shall allow deleting a category, with a confirmation prompt. Deletion shall be blocked if products are assigned to it or if it has child categories.	High
FR-CAT-04	The system shall display categories as a two-level tree (parent → children). Each node shall show its item count.	Medium
FR-CAT-05	A category name must be unique within its level (i.e., two siblings cannot share a name, but a child may share a name with a category under a different parent).	High
FR-CAT-06	The system shall support assigning a color or icon to a category for visual distinction on the sales screen.	Low

3.2 Product / Item Management

The system shall provide full CRUD functionality for product items in the catalog.

ID	Description	Priority
FR-PRD-01	The system shall allow creating a new product with the following fields: Name (required), Category (required), Price (required), Discount Price (optional), Barcode (optional — can be manually entered or auto-generated), Description (optional), Stock Quantity (optional).	High
FR-PRD-02	The system shall allow editing all fields of an existing product.	High
FR-PRD-03	The system shall allow deleting a product with a confirmation prompt.	High
FR-PRD-04	The system shall display a searchable and filterable product list (by name, category, or barcode).	High
FR-PRD-05	If no barcode is provided during creation, the system shall offer to auto-generate a unique Code128 barcode for the product.	High
FR-PRD-06	The system shall allow entering an existing barcode manually if the product already has one.	High

ID	Description	Priority
FR-PRD-07	The system shall be able to print a barcode label for any product using the built-in printer.	Medium
FR-PRD-08	The system shall validate that barcode values are unique across all products.	High
FR-PRD-09	The system shall display the effective selling price, accounting for any discount.	Medium

3.3 Sales / Checkout Module

The sales module is the primary interface used during customer transactions.

ID	Description	Priority
FR-SAL-01	The system shall provide a sales screen where the employee can build a cart of items.	High
FR-SAL-02	The system shall allow adding items to the cart by scanning a barcode using the built-in scanner.	High
FR-SAL-03	The system shall allow adding items to the cart by manually browsing and selecting from the product catalog.	High
FR-SAL-04	The system shall display each cart item with: name, quantity, unit price, applied discount, and line total.	High
FR-SAL-05	The system shall allow changing the quantity of any item in the cart.	High
FR-SAL-06	The system shall allow removing individual items from the cart.	High
FR-SAL-07	The system shall calculate and display: subtotal, total discount, and grand total in real time.	High
FR-SAL-08	The system shall allow applying a transaction-level discount (percentage or fixed amount).	Medium
FR-SAL-09	The system shall allow recording the amount tendered by the customer and display the change due.	High
FR-SAL-10	Upon completing a sale, the system shall save the transaction to the SQLite database with a timestamp.	High
FR-SAL-11	If a scanned barcode does not match any product, the system shall display a clear 'Product not found' notification.	High
FR-SAL-12	The system shall support voiding or canceling a current transaction before it is finalized.	Medium

3.4 Receipt / Invoice Printing

ID	Description	Priority
FR-PRT-01	Upon completing a sale, the system shall automatically print a receipt using the built-in 80mm thermal printer.	High
FR-PRT-02	The receipt shall include: store name/header, date and time, transaction ID, list of items (name, qty, unit price, line total), subtotal, discount, grand total, amount tendered, and change.	High
FR-PRT-03	The system shall allow reprinting the last receipt or any historical receipt.	Medium
FR-PRT-04	The system shall support printing barcode labels (product name + barcode) on demand from the product management screen.	Medium
FR-PRT-05	Printed receipts shall be formatted to fit 80mm thermal paper width.	High
FR-PRT-06	The system shall handle printer errors gracefully (e.g., paper out, printer offline) and notify the user.	High

3.5 Dashboard and Analytics

ID	Description	Priority
FR-DAS-01	The dashboard shall display a summary of key metrics: total sales today, total transactions today, best-selling product today.	High
FR-DAS-02	The system shall display a chart of sales over time (daily/weekly/monthly view).	Medium
FR-DAS-03	The system shall display a breakdown of sales by category.	Medium
FR-DAS-04	The system shall display a list of top-selling products by quantity and by revenue for a selected period.	Medium
FR-DAS-05	All dashboard metrics shall be filterable by a custom date range (start date – end date).	High

3.6 Report Generation

ID	Description	Priority
FR-RPT-01	The system shall allow generating a sales report for a user-defined date range.	High
FR-RPT-02	The report shall include: total revenue, total number of transactions, average transaction value, itemized product sales (quantity sold + revenue per product), sales per category.	High
FR-RPT-03	The system shall export the report as a PDF file saved to the device's storage.	High

ID	Description	Priority
FR-RPT-04	The system shall allow printing the generated report via the built-in printer.	Medium
FR-RPT-05	The system shall allow sharing the PDF report via Android's share intent (email, messaging, file manager).	Low

3.7 Purchase Orders (Stock Receiving)

The system shall allow the administrator to record items received into inventory via a purchase order entry. This updates each item's stock quantity without requiring a separate stock-count screen. A purchase order captures what was bought, at what cost price, and in what quantity, giving a simple procurement history.

ID	Description	Priority
FR-PO-01	The system shall provide a 'New Purchase Order' form accessible from the Dashboard.	High
FR-PO-02	The form shall allow the user to add one or more line items. Each line specifies: Product (selected from existing DB items via search or barcode scan), Purchase Quantity (required, integer > 0), and Purchase Price per unit (required — the cost price paid to the supplier, stored for reference only and not affecting the selling price).	High
FR-PO-03	Upon saving a purchase order, the system shall increment each listed product's stock_qty in the products table by the specified purchase quantity.	High
FR-PO-04	Each saved purchase order shall be stored persistently with: auto-generated PO number, creation timestamp, optional supplier note, and the list of received lines.	High
FR-PO-05	The system shall display a purchase order history list showing PO number, date, number of items received, and total cost value.	Medium
FR-PO-06	The user shall be able to view the detail of any past purchase order (read-only). Purchase orders shall not be editable after saving to preserve stock integrity.	Medium
FR-PO-07	The system shall validate that quantity is a positive integer and purchase price is a non-negative value before allowing the order to be saved.	High

3.8 Multi-Currency Support

The system supports one primary currency and one secondary currency. Prices are always stored in the primary currency. The secondary currency is displayed as a convenience equivalent, calculated using a manually configured exchange rate. The exchange rate in effect at the moment of

each sale is snapshotted into the transaction record so that historical reports remain accurate regardless of future rate changes.

ID	Description	Priority
FR-CUR-01	The administrator shall be able to configure a primary currency (code, symbol, e.g. USD / $) in the application settings.	**High**
FR-CUR-02	The administrator shall be able to configure one secondary currency (code, symbol) and enter its current exchange rate relative to the primary currency via a dedicated input field in settings (e.g., 1 USD = 3,660 IQD).	**High**
FR-CUR-03	All item prices, discounts, and totals shall be stored in the primary currency only.	**High**
FR-CUR-04	The POS screen and item listings shall display each monetary value in both currencies simultaneously (e.g., $5.00 / 18,300 IQD), computed from the current exchange rate.	**High**
FR-CUR-05	When a sale is completed, the system shall snapshot the active exchange rate into the transaction record alongside all monetary amounts.	**High**
FR-CUR-06	All generated reports shall include both the primary-currency totals and the secondary-currency equivalents using the rate that was active at the time of each transaction, not the current rate.	**High**
FR-CUR-07	Receipts shall print monetary amounts in both currencies using the exchange rate at the time of the sale.	**Medium**
FR-CUR-08	The system shall display a visible indicator on the POS screen and Dashboard showing the currently active exchange rate (e.g., "Rate: 1 USD = 3,660 IQD — last updated [date]").	**Medium**
FR-CUR-09	The exchange rate field shall accept decimal values. The system shall warn the administrator if the entered rate is 0 or negative before saving.	**High**

4. Non-Functional Requirements

4.1 Performance

ID	Description	Priority
NFR-PRF-01	The application shall launch and be ready for use within 3 seconds on the target device.	High
NFR-PRF-02	Barcode scanning to item-added-in-cart response time shall be under 500ms.	High
NFR-PRF-03	Receipt printing shall complete within 5 seconds of transaction confirmation.	High
NFR-PRF-04	Dashboard analytics shall load within 2 seconds for data sets up to 100,000 transactions.	Medium

4.2 Reliability

ID	Description	Priority
NFR-REL-01	The application shall not lose transaction data in the event of a crash or unexpected shutdown.	High
NFR-REL-02	SQLite transactions shall use ACID-compliant operations to prevent data corruption.	High
NFR-REL-03	The application shall handle hardware errors (printer, scanner) without crashing.	High

4.3 Usability

ID	Description	Priority
NFR-USB-01	The UI shall be optimized for touch on an 8-inch screen with a minimum touch target size of 48dp.	High
NFR-USB-02	All critical actions (add item, checkout, print) shall be reachable within 3 taps from the main screen.	Medium
NFR-USB-03	The application shall provide clear visual and audio feedback for barcode scan events (success/failure).	Medium
NFR-USB-04	The application shall support both portrait and landscape orientations.	Low

4.4 Security

ID	Description	Priority
NFR-SEC-01	Access to the Dashboard / Admin module shall require a PIN or password.	High
NFR-SEC-02	The SQLite database file shall be stored in the app's private internal storage, inaccessible to other apps.	High
NFR-SEC-03	Sensitive configuration (e.g., store name, PIN) shall not be stored in plain text.	Medium

4.5 Maintainability

ID	Description	Priority
NFR-MNT-01	The codebase shall follow a clear architectural pattern (e.g., MVVM or Clean Architecture).	Medium
NFR-MNT-02	Database schema migrations shall be handled automatically on app update.	High
NFR-MNT-03	The application shall write error logs to a local log file for debugging.	Medium

4.6 Portability

ID	Description	Priority
NFR-PRT-01	The application shall function fully without internet connectivity.	High

5. Use Cases

UC-01: Process a Sale

Use Case ID	UC-01
Title	Process a Sale
Primary Actor	Employee / Cashier
Description	The cashier scans or selects items, completes a transaction, and prints a receipt.
Preconditions	The sales screen is open. At least one product exists in the database.
Main Flow	1. Cashier scans item barcode OR browses catalog and selects item. 2. Item is added to cart with quantity 1. 3. Repeat step 1-2 for all items. 4. Cashier taps 'Checkout'. 5. System displays total. Cashier enters amount tendered. 6. System calculates and displays change. 7. Cashier confirms sale. System saves transaction. 8. Printer automatically prints the receipt.
Postconditions	Transaction is saved to SQLite. Receipt is printed. Cart is cleared.
Exceptions / Alt.	UC-01a: Barcode not found – system shows 'Product not found' alert. UC-01b: Printer offline – system notifies user and saves transaction without printing.

UC-02: Add a New Product

Use Case ID	UC-02
Title	Add a New Product
Primary Actor	Administrator
Description	The admin creates a new product entry with pricing, category, and barcode information.
Preconditions	Admin is logged into the Dashboard module.
Main Flow	1. Admin navigates to Products > Add New. 2. Admin fills in: Name, Category, Price, Discount Price (optional). 3. Admin enters an existing barcode OR taps 'Generate Barcode'. 4. System generates a unique Code128 barcode. 5. Admin saves the product. 6. System validates uniqueness of barcode and saves to SQLite.
Postconditions	New product is saved and appears in the product list and sales catalog.
Exceptions / Alt.	UC-02a: Duplicate barcode – system rejects entry and prompts correction.

UC-03: Generate Sales Report

Use Case ID	UC-03
Title	Generate Sales Report
Primary Actor	Administrator
Description	The admin generates a PDF report of sales for a selected date range.
Preconditions	Admin is on the Dashboard. Sales data exists for the selected period.
Main Flow	1. Admin navigates to Reports. 2. Admin selects start date and end date. 3. Admin taps 'Generate Report'. 4. System queries SQLite for matching transactions. 5. System compiles and displays the report summary. 6. Admin taps 'Export PDF' or 'Print'. 7. System saves PDF to device storage and/or sends to printer.
Postconditions	PDF file is saved to device. Report is optionally printed.
Exceptions / Alt.	UC-03a: No data in range – system displays 'No sales found for this period'.

UC-04: Print Barcode Label

Use Case ID	UC-04
Title	Print Barcode Label
Primary Actor	Administrator
Description	The admin prints a barcode label for a product directly from the product management screen.
Preconditions	Product exists in the database and has a barcode assigned.
Main Flow	1. Admin opens the product from the product list. 2. Admin taps 'Print Label'. 3. System sends ESC/POS print job with product name and barcode image. 4. Printer outputs the label.
Postconditions	Barcode label is printed on the thermal printer.
Exceptions / Alt.	UC-04a: Printer offline – system shows error message.

UC-05: Record a Purchase Order

Use Case ID	UC-05
Title	Record a Purchase Order
Primary Actor	Administrator
Description	The admin records a stock delivery, specifying items received, quantities, and cost prices. The syst
Preconditions	Admin is logged into the Dashboard. At least one product exists in the database.
Main Flow	1. Admin navigates to Purchase Orders > New PO. 2. Admin adds line items: searches/scans each product, enters quantity received and purchase pric 3. Admin repeats step 2 for all received products. 4. Admin optionally enters a supplier note. 5. Admin taps 'Save Purchase Order'. 6. System validates all fields (qty > 0, price >= 0). 7. System saves the PO record and increments stock_qty for each product. 8. System displays confirmation with the generated PO number.
Postconditions	PO saved; stock quantities updated in products table; PO appears in history list.
Exceptions / Alt.	UC-05a: Invalid quantity/price – system highlights the offending field and blocks save. UC-05b: Product not found via scan – system shows 'Product not found' and allows manual search

6. Data Model

The following SQLite table schemas define the core data structures of the application.

6.1 categories

```
CREATE TABLE categories (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    parent_id   INTEGER REFERENCES categories(id) ON DELETE SET NULL,
    name        TEXT    NOT NULL,
    description TEXT,
    color       TEXT,
    created_at  TEXT    NOT NULL DEFAULT (datetime('now')),
    updated_at  TEXT    NOT NULL DEFAULT (datetime('now')),
    UNIQUE(name, parent_id)  -- name unique within same parent level
);
-- parent_id NULL = root category. App should enforce max 2 levels of nesting.
```

6.2 products

```
CREATE TABLE products (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    category_id    INTEGER NOT NULL REFERENCES categories(id),
    name           TEXT    NOT NULL,
    barcode        TEXT    UNIQUE,
    price          REAL    NOT NULL CHECK(price >= 0),
    discount_price REAL             CHECK(discount_price >= 0),
    description    TEXT,
    stock_qty      INTEGER DEFAULT 0,
    created_at     TEXT    NOT NULL DEFAULT (datetime('now')),
    updated_at     TEXT    NOT NULL DEFAULT (datetime('now'))
);
```

6.3 transactions

```
CREATE TABLE transactions (
    id                      INTEGER PRIMARY KEY AUTOINCREMENT,
    total_amount            REAL    NOT NULL,
    discount_amount         REAL    NOT NULL DEFAULT 0,
    grand_total             REAL    NOT NULL,
    amount_tendered         REAL    NOT NULL,
    change_due              REAL    NOT NULL,
    primary_currency_code   TEXT    NOT NULL,
    secondary_currency_code TEXT,
    exchange_rate_snapshot  REAL,   -- 1 primary = N secondary at sale time
    note                    TEXT,
    created_at              TEXT    NOT NULL DEFAULT (datetime('now'))
);
```

6.4 transaction_items

```
CREATE TABLE transaction_items (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    transaction_id INTEGER NOT NULL REFERENCES transactions(id),
    product_id     INTEGER NOT NULL REFERENCES products(id),
    product_name   TEXT    NOT NULL,  -- snapshot at time of sale
    unit_price     REAL    NOT NULL,  -- primary currency
    discount_price REAL,              -- primary currency (NULL if no discount)
    quantity       INTEGER NOT NULL CHECK(quantity > 0),
    line_total     REAL    NOT NULL   -- primary currency
    -- Secondary-currency equivalent = line_total * transactions.exchange_rate_
snapshot
);
```

6.5 purchase_orders

```
CREATE TABLE purchase_orders (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    po_number   TEXT    NOT NULL UNIQUE,  -- e.g. 'PO-000042'
    supplier_note TEXT,
    total_cost  REAL    NOT NULL DEFAULT 0,
    created_at  TEXT    NOT NULL DEFAULT (datetime('now'))
);
```

6.6 purchase_order_items

```
CREATE TABLE purchase_order_items (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    purchase_order_id INTEGER NOT NULL REFERENCES purchase_orders(id),
    product_id        INTEGER NOT NULL REFERENCES products(id),
    product_name      TEXT    NOT NULL,  -- snapshot at time of PO
    quantity          INTEGER NOT NULL CHECK(quantity > 0),
    purchase_price    REAL    NOT NULL CHECK(purchase_price >= 0),
    line_cost         REAL    NOT NULL   -- quantity * purchase_price
);
-- After insert, trigger or application code must execute:
-- UPDATE products SET stock_qty = stock_qty + :quantity WHERE id = :product_id
```

6.7 settings

```
CREATE TABLE settings (
    key   TEXT PRIMARY KEY,
    value TEXT
);
-- Currency-related keys:
-- 'primary_currency_code'    e.g. 'USD'
-- 'primary_currency_symbol'  e.g. '$'
-- 'secondary_currency_code'  e.g. 'EUR'
-- 'secondary_currency_symbol' e.g. 'EUR'
-- 'exchange_rate'            e.g. '0.92' (1 primary = N secondary)
-- 'exchange_rate_updated_at' ISO-8601 timestamp of last manual rate update
-- Other keys: 'store_name', 'admin_pin_hash', 'receipt_footer', 'receipt_show_
qr'
```

Note: *product_name* and all pricing fields are snapshotted into *transaction_items* at the time of sale to preserve historical accuracy even if the product is later edited or deleted. Likewise, the *exchange_rate_snapshot* on the *transactions* table captures the rate active at sale time; reports always use this stored rate, never the current live rate, so historical totals in secondary currency remain consistent.

7. Hardware Interfaces

7.1 Built-in Barcode Scanner

The POS-T8-3 includes an integrated barcode scanner. The application shall interface with it using one of the following mechanisms (to be confirmed against the device vendor SDK):

- **HID / Keyboard emulation** – Scanner output is treated as keyboard input. The app listens to a hidden input field that receives the scanned string followed by a newline character. This is the most portable approach.
- **Vendor SDK / Intent broadcast** – The device vendor may provide an Android SDK or broadcast Intent for scanner events. This should be preferred if available, as it offers more control (e.g., disabling the scanner, configuring symbologies).

Supported barcode symbologies shall include at minimum:

- Code128
- Code39
- EAN-13
- EAN-8
- QR Code
- UPC-A

7.2 Built-in Thermal Printer

The built-in 80mm thermal printer shall be accessed via the ESC/POS protocol. The application shall use a Java/Kotlin ESC/POS library or the device vendor's printer SDK to construct and send print jobs.

Required ESC/POS capabilities:

- Text formatting: bold, normal, font size adjustment.
- Alignment: left, center, right.
- Line feed and partial cut commands.
- Barcode printing: Code128 on 80mm width.
- QR Code printing (for optional receipt QR).

The application shall gracefully handle printer states: offline, paper out, and busy. Failed print jobs shall be logged and the user shall be notified.

8. Appendix – Screen Module Summary

Screen / Module	Primary Purpose	Access
Sales Screen	Build cart, scan/select items, checkout	All users
Product List	Browse, search, and manage products	Admin
Add / Edit Product	CRUD form for product data and barcode	Admin
Category Manager	CRUD for product categories	Admin
Purchase Orders	Record stock purchases; auto-update inventory	Admin
Dashboard	Overview KPI metrics and charts	Admin (view: all)
Reports	Date-range report generation and PDF export	Admin
Settings	Store name, PIN, currency, printer config	Admin
Receipt Preview	Preview before printing, reprint past receipts	All users

9. Revision History

Version	Date	Author	Description
1.0	2026-06-07	Adones	Initial draft